Fishing Game Static Analysis and Refactoring

1 Script overview

This CLI fishing game lets players explore, collect items and restore stamina by resting (slow life) Finding treasure wins the game but you need to be careful , running out of stamina ends it.

2 Static analysis results

Pylint checks coding conventions, suspicious constructs and maintainability. The supplied runs show the following progression:

Stage	Result
Initial script	6.25/10
After formatting	8.20/10
After manual corrections	10.00/10

Pylint scores improved from **6.25/10** initially to **8.20/10** after formatting and **10.00/10** after manual corrections.

Mypy reported: **“Success: no issues found in 1 source file.”** No initial mypy result was recorded. Mypy checks type compatibility, but untyped code may receive limited checking.

3 Findings and code improvements

Readability: Indentation, long lines and whitespace were corrected. Docstrings were added to explain the module and functions. (W0311 , C0301 , C0114/C0116)

Consistency: Return types were made consistent. The sleep variable became energy_message to avoid name shadowing and separate text from numeric stamina. An unnecessary f-string, redundant pass and unused parameters were removed.

Structure: A GameState dataclass replaced positional state access. Room uses TypedDict to define room data, and function annotations support type checking. default_factory=list gives each player an independent inventory

BEFORE

```
def sleep_action(game_state):
    game_state[5] = 100
```

AFTER

```
def sleep_action(game_state: GameState) -> None:
    """Recover your stamina"""
    game_state.stamina = 100
```

The named attribute makes the update clearer. Additional fixes corrected room destinations, handled both treasure items and prevented removal of missing items.

4 Comparison and reflection

Mypy checks types, while Pylint checks style and code quality. Both improve maintainability, but neither guarantees correct gameplay. Runtime checks are still needed.